

Exception Handling in Spring Boot

Overview

Exception handling is one of the most important parts of building professional REST APIs. A good API should not only return data; it should also clearly communicate what happened during the request.

A professional API response answers questions like:

- Was the request successful?
- Did the client send something wrong?
- Was the requested resource missing?
- Did something break inside the server?
- Can the frontend understand the response and show the right message?

This is where **HTTP status codes**, **ResponseEntity**, **custom exceptions**, and **global exception handling** become important.

1. API Response: Communication Between Two Systems

An API response usually has three main parts:

1. Status Code
2. Headers
3. Body

Status Code

The status code tells the meaning of the response at the HTTP level.

Example:

```
200 OK
404 Not Found
500 Internal Server Error
```

The frontend, mobile app, or another backend service can read this status code and understand what happened.

Headers

Headers contain metadata about the response.

Example:

```
Location: /api/students/10
Content-Type: application/json
```

Headers are not the main data, but they give extra information about the response.

Body

The body contains the actual data or error message.

Example:

```
{
  "id": 10,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

2. Beginner API vs Professional API

A beginner API usually focuses only on this question:

```
Did I get data or not?
```

A professional API focuses on a better question:

Did the server clearly tell the client what happened?

For example, suppose the frontend calls:

```
GET /api/students/10
```

Many things can happen:

- The student exists.
- The student does not exist.
- The ID format is wrong.
- The database is down.
- There is a bug in backend code.

A professional API should communicate each case properly.

3. Example: Professional API Responses

Case 1: Student Exists

```
200 OK
```

```
{  
  "id": 10,  
  "name": "Rahul",  
  "email": "rahul@gmail.com"  
}
```

Meaning:

The request was successful, and the student data is returned.

Case 2: Student Does Not Exist

404 Not Found

```
{
  "message": "Student with id 10 not found"
}
```

Meaning:

The request was valid, but the requested resource does not exist.

Case 3: Invalid Request

400 Bad Request

```
{
  "message": "Student id must be a number"
}
```

Meaning:

The client sent an invalid request.

Case 4: Server Error

500 Internal Server Error

```
{
  "message": "Something went wrong on the server"
}
```

Meaning:

The client may have sent a valid request, but something failed inside the server.

4. Common Problems in Beginner CRUD APIs

Problem 1: Everything Returns `200 OK`

Many beginner APIs return `200 OK` for almost every operation:

Operation	Beginner Response
Create student	<code>200 OK</code>
Fetch student	<code>200 OK</code>
Update student	<code>200 OK</code>
Delete student	<code>200 OK</code>
Student not found	<code>200 OK</code> with <code>null</code>
Validation failed	Random error response

This is not expressive.

Better responses would be:

Situation	Better Status Code
New resource created	<code>201 Created</code>
Resource not found	<code>404 Not Found</code>
Validation failed	<code>400 Bad Request</code>
Duplicate data	<code>409 Conflict</code>
Unexpected backend issue	<code>500 Internal Server Error</code>

Problem 2: Repeated Error Logic in Controllers

A beginner controller may look like this:

```
@GetMapping("/{id}")
public ResponseEntity<?> getStudentById(@PathVariable Long id) {
    Student student = studentService.getStudentById(id);

    if (student == null) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body("Student not found");
    }

    return ResponseEntity.ok(student);
}
```

This is better than returning `null`, but it creates repeated logic.

The same check may appear in many methods:

```
getStudentById
updateStudent
deleteStudent
getCourseById
getTeacherById
getBatchById
```

Everywhere we write:

```
if (data == null) {
    return 404;
}
```

This creates duplicate code and makes controllers bulky.

Problem 3: Inconsistent Error Responses

One API may return:

```
{
  "message": "Student not found"
}
```

Another API may return:

```
"Course not found"
```

Another API may return:

```
{
  "error": "Something went wrong",
  "status": 500
}
```

Another API may return Spring Boot's default error response.

This inconsistency makes the frontend harder to build.

A professional backend should return a consistent structure for errors.

5. What Happens When an Exception Occurs in Spring Boot?

Think about the normal request flow:

```
Client
↓
Tomcat / Embedded Server
↓
DispatcherServlet
↓
Controller
↓
Service
↓
Repository
```

↓
Database

Now suppose the service layer throws an exception:

```
public Student getStudentById(Long id) {  
    return studentRepository.findById(id).get();  
}
```

If the student does not exist, `findById(id)` returns an empty `Optional`.

Calling `.get()` on an empty `Optional` throws an exception.

The exception then travels backward:

```
Repository / Service exception  
↓  
Controller does not handle it  
↓  
Spring MVC tries to resolve it  
↓  
If no custom handler exists, Spring Boot default error handling takes over
```

Spring Boot does have default error handling, but the default response may not match the meaning of your business case.

6. Spring Boot Default Error Handling

Spring Boot provides a default `/error` mapping.

When an unhandled error occurs, Spring Boot internally forwards the request to `/error`.

For browser clients, Spring Boot may show the Whitelabel Error Page.

For API clients such as Postman, it usually returns a JSON error response.

Test Example

Create a test endpoint:

Coder Army

```
@GetMapping("/test/500")
public String test500Error() {
    throw new RuntimeException("Something went wrong inside s
erver");
}
```

Open this URL in the browser:

```
http://localhost:8080/test/500
```

You may see an HTML page similar to:

```
Whitelabel Error Page
This application has no explicit mapping for /error, so you are seeing this as a fallback.
```

In Postman, the response may look like this:

```
{
  "timestamp": "2026-07-05T11:45:06.511Z",
  "status": 500,
  "error": "Internal Server Error",
  "path": "/test/500"
}
```

By default, Spring Boot hides detailed exception messages for safety.

For learning or local testing, you can temporarily add this in `application.properties` :

```
spring.web.error.include-message=always
```

Then the response may include the exception message:

```
{
  "timestamp": "2026-07-05T10:15:30.123+00:00",
```

```
"status": 500,  
"error": "Internal Server Error",  
"message": "Something went wrong inside server",  
"path": "/test/500"  
}
```

Do not expose internal exception messages in production. They may reveal implementation details.

7. Problem with Default Error Response

Default error handling is useful, but it is not always domain-specific.

Suppose the real issue is:

```
Student with id 2 does not exist.
```

But the response says:

```
{  
  "status": 500,  
  "error": "Internal Server Error"  
}
```

This is misleading.

The server did not fail because of an unknown internal crash. The requested student simply does not exist.

The correct response should be:

```
404 Not Found
```

This is one of the main reasons we need custom exception handling.

8. HTTP Status Codes for REST APIs

The response body is for data and human-readable messages.

The status code is for protocol-level meaning.

Frontend code can use status codes like this:

```
if (response.status === 404) {
    showNotFoundMessage();
}
```

HTTP status codes are grouped into categories:

Category	Meaning
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client error
5xx	Server error

For a beginner-to-intermediate Spring Boot CRUD API, these status codes are enough:

```
200 OK
201 Created
204 No Content
400 Bad Request
404 Not Found
409 Conflict
500 Internal Server Error
```

9. Common Status Codes in CRUD APIs

Situation	Status Code
Data fetched successfully	200 OK
Data updated successfully	200 OK

Situation	Status Code
New resource created	201 Created
Delete successful with no body	204 No Content
Invalid request body or validation failure	400 Bad Request
Resource does not exist	404 Not Found
Duplicate email, username, or roll number	409 Conflict
Unexpected backend failure	500 Internal Server Error

10. 201 Created

Use **201 Created** when a new resource is successfully created.

Example:

```
POST /api/students
```

Response:

```
201 Created
```

```
{
  "id": 10,
  "name": "Riya",
  "email": "riya@gmail.com"
}
```

This tells the client:

```
A new student resource has been created successfully.
```

11. 204 No Content

204 No Content means the request was successful, but there is no response body.

This is commonly used for delete APIs.

Example:

```
DELETE /api/students/1
```

Response:

```
204 No Content
```

No body is returned.

For delete operations, both approaches are common:

```
204 No Content without body
200 OK with message/body
```

For clean REST APIs, **204 No Content** is often preferred.

12. **409 Conflict**

409 Conflict means the request could not be completed because it conflicts with the current state of the resource.

Example:

A student already exists with this email:

```
{
  "id": 1,
  "email": "amit@gmail.com"
}
```

Now the client sends:

POST /api/students

```
{
  "name": "Another Amit",
  "email": "amit@gmail.com",
  "age": 23
}
```

The JSON is valid.

The email format is valid.

But the email already exists.

So the correct response is:

409 Conflict

```
{
  "message": "Student with email amit@gmail.com already exist
s"
}
```

Use **409 Conflict** for cases like:

- Duplicate email
- Duplicate username
- Duplicate roll number
- Trying to create something that already exists
- Version conflict during update
- Business rule conflict

13. **500 Internal Server Error**

500 Internal Server Error means something unexpected failed inside the backend.

Examples:

- Database connection failed
- NullPointerException due to a bug
- Unexpected runtime exception
- Third-party service failure
- File system error
- Server configuration issue

Use **500** for unexpected server-side failures, not for normal business cases like "student not found".

ResponseEntity in Spring Boot

14. Why Use **ResponseEntity** ?

In a basic controller, we may write:

```
@GetMapping("/{id}")
public Student getStudentById(@PathVariable Long id) {
    return studentService.getStudentById(id);
}
```

This returns only the body.

Spring decides the status code automatically.

With **ResponseEntity**, we control the response explicitly:

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
    Student student = studentService.getStudentById(id);
```

```
return ResponseEntity.ok(student);  
}
```

This means:

```
Response body: student  
Status code: 200 OK
```

The important difference is control.

Without `ResponseEntity`:

```
Spring decides the response status.
```

With `ResponseEntity`:

```
We explicitly decide the response status, headers, and body.
```

15. Mental Model of ResponseEntity

`ResponseEntity` can represent:

```
Body + Status  
Body + Headers + Status  
Status only
```

Examples:

```
ResponseEntity.ok(student);
```

```
200 OK + student body
```

```
ResponseEntity.status(HttpStatus.CREATED).body(savedStudent);
```

201 Created + saved student body

```
ResponseEntity.noContent().build();
```

204 No Content + no body

16. Returning Headers with ResponseEntity

Headers contain extra metadata.

For a create API, the `Location` header is useful.

Example:

```
Location: /api/students/10
```

This tells the client where the newly created student can be found.

```
@PostMapping
public ResponseEntity<Student> createStudent(@RequestBody Student student) {
    Student savedStudent = studentService.createStudent(student);

    URI location = URI.create("/api/students/" + savedStudent.getId());

    return ResponseEntity
        .created(location)
```

```
}  
    .body(savedStudent);  
}
```

Response:

```
201 Created  
Location: /api/students/10
```

```
{  
  "id": 10,  
  "name": "Riya",  
  "email": "riya@gmail.com"  
}
```

Here:

```
Status → 201 Created  
Header → Location: /api/students/10  
Body   → saved student data
```

`ResponseEntity.created(location)` automatically sets:

```
Status: 201 Created  
Header: Location
```

17. Common ResponseEntity Methods

`ResponseEntity.ok()`

```
return ResponseEntity.ok(student);
```

Meaning:

Status code: 200 OK
Body: student

It is a shortcut for:

```
return ResponseEntity  
    .status(HttpStatus.OK)  
    .body(student);
```

ResponseEntity.noContent()

Use this when the operation is successful but there is no body to return.

Common use case:

DELETE API

```
@DeleteMapping("/{id}")  
public ResponseEntity<Void> deleteStudent(@PathVariable Long  
id) {  
    studentService.deleteStudent(id);  
  
    return ResponseEntity.noContent().build();  
}
```

Meaning:

Status code: 204 No Content
Body: nothing

Why Error Handling Should Not Live Inside Controllers

18. Problem with Handling Errors in Controller

Once beginners learn `ResponseEntity`, they often start writing all error handling inside the controller.

```
@GetMapping("/{id}")
public ResponseEntity<?> getStudentById(@PathVariable Long id) {
    Student student = studentService.getStudentById(id);

    if (student == null) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body("Student not found");
    }

    return ResponseEntity.ok(student);
}
```

This looks fine for one method.

But in a real project, the controller starts doing too many things:

1. Receive request
2. Call service
3. Check null
4. Decide error status
5. Decide error message
6. Build error response
7. Build success response

A controller should mainly do this:

1. Receive request
2. Call service
3. Return response

Business checks should stay in the service layer, and common error response logic should stay in one central place.

19. Example: Duplicate Email Check in Controller

A controller may become like this:

```
@PostMapping
public ResponseEntity<?> createStudent(@RequestBody Student student) {
    boolean emailExists = studentService.existsByEmail(student.getEmail());

    if (emailExists) {
        return ResponseEntity
            .status(HttpStatus.CONFLICT)
            .body("Email already exists");
    }

    Student savedStudent = studentService.createStudent(student);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(savedStudent);
}
```

Now the controller has too much `if-else` logic.

This same pattern will repeat across many APIs.

That is not clean design.

20. Problems Caused by Repeated Error Logic

1. Repetition

The same error handling logic appears again and again.

2. Less Readable Controllers

Controllers become full of `if-else` blocks.

3. Weak Service Layer Signals

If the service returns `null`, the controller has to guess what happened.

4. Inconsistent Error Responses

Different developers may return different error structures.

5. Poor Separation of Concerns

The controller should not contain repeated business error response logic.

21. Better Separation of Responsibility

A cleaner design looks like this:

Layer	Responsibility
Controller	Receive request and return success response
Service	Run business logic and throw meaningful exceptions
Global Exception Handler	Convert exceptions into proper HTTP responses

This gives a clean flow:

```

Request comes
↓
Controller calls service
↓
Service either returns data or throws exception
↓
GlobalExceptionHandler catches exception
↓
Proper error response goes back to client
  
```

Global Exception Handling in Spring Boot

22. What Is Global Exception Handling?

When an API fails, two things happen:

1. Some exception occurs inside backend code.
2. That exception must be converted into a proper HTTP response.

Global exception handling means:

Instead of handling errors separately inside every controller, we create one central place that catches exceptions and converts them into proper API responses.

Clean flow:

```
Client
↓
Controller
↓
Service
↓
Exception thrown
↓
GlobalExceptionHandler catches exception
↓
HTTP response returned
```

23. What Is `@ControllerAdvice` ?

`@ControllerAdvice` is a Spring annotation used to define common behavior for multiple controllers.

In simple words:

`@ControllerAdvice` allows one class to provide common support to controllers.

That support can include:

- Exception handling
- Model attributes
- Data binding configuration

Example:

```
@ControllerAdvice
public class GlobalExceptionHandler {

}
```

By itself, `@ControllerAdvice` does not handle exceptions.

To actually handle exceptions, we need methods annotated with:

```
@ExceptionHandler
```

So the combination is:

```
@ControllerAdvice + @ExceptionHandler
```

24. What Is `@RestControllerAdvice` ?

In REST APIs, we usually return JSON responses.

For that, Spring provides:

```
@RestControllerAdvice
```

`@RestControllerAdvice` is a shortcut for:

```
@ControllerAdvice
@ResponseBody
```

That means exception handler methods return response content directly as JSON instead of resolving a view or JSP page.

For REST API projects, prefer:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

}
```

25. What Is `@ExceptionHandler` ?

`@ExceptionHandler` is used on a method to handle a specific exception type.

Example:

```
@ExceptionHandler(RuntimeException.class)
public ResponseEntity<String> handleRuntimeException(RuntimeException ex) {
    return ResponseEntity
        .status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(ex.getMessage());
}
```

Meaning:

If RuntimeException occurs, call this method and return its response.

The exception class decides which handler method will run.

Spring tries to use the most specific matching exception handler.

For example, if you have handlers for both `ResourceNotFoundException` and `Exception`, then `ResourceNotFoundException` will be handled by its specific handler, not the generic `Exception` handler.

26. Creating a Global Exception Handler Class

Recommended package:

```
in.strikes.dtoDemo.exception
```

Example:

```
package in.strikes.dtoDemo.exception;

import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

}
```

This class becomes the central place for API error handling.

27. Handling Generic Exceptions

Generic exception means:

```
Any unexpected error that we did not specifically handle.
```

Example:

```
@ExceptionHandler(Exception.class)
public ResponseEntity<String> handleGenericException(Exception ex) {
    return ResponseEntity
        .status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body("Something went wrong. Please try again lat
```

```
er.");  
}
```

This generic handler is a safety net.

It should not be used for normal business cases like:

```
Student not found  
Duplicate email  
Validation failed
```

Those should have specific exception handlers.

The generic handler is for unexpected errors.

Custom Exceptions

28. Why Create Custom Exceptions?

Java already has generic exceptions like:

```
RuntimeException  
IllegalArgumentException  
NoSuchElementException
```

But these do not clearly describe the business meaning.

For example:

```
RuntimeException
```

does not tell much.

But this:

```
ResourceNotFoundException
```

clearly says:

The requested resource does not exist.

Custom exceptions make service-layer code more meaningful.

29. Creating `ResourceNotFoundException`

```
package in.strikes.dtoDemo.exception;

public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

Why extend `RuntimeException` ?

In Spring Boot service-layer business errors, unchecked exceptions are commonly used.

That means you do not need to write `throws` in every method signature.

For beginner Spring Boot projects, extending `RuntimeException` is simple and practical.

30. Throwing Custom Exception from Service Layer

Instead of returning `null`, the service should clearly say what happened.

Bad approach:

```
public Student getStudentById(Long id) {
    return studentRepository.findById(id).orElse(null);
}
```

Better approach:

```
public Student getStudentById(Long id) {  
    return studentRepository.findById(id)  
        .orElseThrow(() -> new ResourceNotFoundException(  
            "Student with id " + id + " not found"  
        ));  
}
```

Now the service behavior is clear:

```
If student exists, return the student.  
If student does not exist, throw ResourceNotFoundException.
```

This is much better than returning `null`.

31. Clean Controller After Using Custom Exception

Once the service throws a meaningful exception, the controller becomes clean:

```
@GetMapping("/{id}")  
public ResponseEntity<Student> getStudentById(@PathVariable Long id) {  
    Student student = studentService.getStudentById(id);  
  
    return ResponseEntity.ok(student);  
}
```

There is no `if`.

There is no `null` check.

There is no `404` logic.

The success response stays in the controller.

The error response goes to the global exception handler.

32. Update API Example

Service

```
public Student updateStudent(Long id, Student updatedStudent)
{
    Student existingStudent = studentRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException(
            "Student with id " + id + " not found"
        ));

    existingStudent.setName(updatedStudent.getName());
    existingStudent.setEmail(updatedStudent.getEmail());
    existingStudent.setAge(updatedStudent.getAge());

    return studentRepository.save(existingStudent);
}
```

Controller

```
@PutMapping("/{id}")
public ResponseEntity<Student> updateStudent(
    @PathVariable Long id,
    @RequestBody Student student
) {
    Student updatedStudent = studentService.updateStudent(id,
        student);

    return ResponseEntity.ok(updatedStudent);
}
```

33. Delete API Example

Service

```

public void deleteStudent(Long id) {
    Student existingStudent = studentRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException(
            "Student with id " + id + " not found"
        ));

    studentRepository.delete(existingStudent);
}

```

Controller

```

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteStudent(@PathVariable Long
id) {
    studentService.deleteStudent(id);

    return ResponseEntity.noContent().build();
}

```

Flow:

```

If deletion succeeds → controller returns 204 No Content
If student does not exist → service throws ResourceNotFoundException
GlobalExceptionHandler returns 404 Not Found

```

34. Handling Custom Exception Globally

Now handle `ResourceNotFoundException` in `GlobalExceptionHandler`.

```

package in.strikes.dtoDemo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandl

```

```

er;
import org.springframework.web.bind.annotation.RestController
Advice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleResourceNotFoundExcep
tion(
        ResourceNotFoundException ex
    ) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGenericException(Exce
ption ex) {
        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Something went wrong. Please try again
later.");
    }
}

```

This works, but returning plain strings is not the most professional structure. Next, we improve it using an error response DTO.

Standard Error Response DTO

35. Why Use an Error Response DTO?

An exception is a Java-side problem.

Example:

```
throw new ResourceNotFoundException("Student with id 100 not found");
```

The client does not understand Java exceptions.

The client understands HTTP responses.

So our job is:

```
Java exception
↓
GlobalExceptionHandler
↓
Standard error response JSON
```

Instead of returning only a string, we can return a structured error response.

36. ErrorResponseDto Structure

```
package in.strikes.dtoDemo.dto;

import java.time.LocalDateTime;

public class ErrorResponseDto {

    private LocalDateTime timestamp;
    private int statusCode;
    private String error;
    private String message;
    private String path;

    public ErrorResponseDto(
        LocalDateTime timestamp,
        int statusCode,
        String error,
```

```
        String message,
        String path
    ) {
        this.timestamp = timestamp;
        this.statusCode = statusCode;
        this.error = error;
        this.message = message;
        this.path = path;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public int getStatusCode() {
        return statusCode;
    }

    public String getError() {
        return error;
    }

    public String getMessage() {
        return message;
    }

    public String getPath() {
        return path;
    }
}
```

Example response:

```
{
  "timestamp": "2026-07-07T10:45:30",
  "statusCode": 404,
```

```
"error": "Not Found",
"message": "Student with id 100 not found",
"path": "/api/students/100"
}
```

37. Meaning of Each Error Field

Field	Meaning
<code>timestamp</code>	When the error happened
<code>statusCode</code>	HTTP status code returned
<code>error</code>	Short HTTP error name
<code>message</code>	Specific explanation of the failure
<code>path</code>	API endpoint where the error occurred

Why `timestamp` is useful

It helps backend developers match the API error with logs.

Why `statusCode` is useful

Frontend code can use it programmatically.

Example:

```
if (error.statusCode === 404) {
    showNotFoundPage();
}
```

Why `path` is useful

If one page calls multiple APIs, the error response itself tells which endpoint failed.

Example:

```
/api/students/100
/api/courses/5
```

```
/api/teachers/8
```

38. Getting Request Path Using `HttpServletRequest`

In Spring Boot, we can get the current request path using:

```
request.getRequestURI()
```

Example handler:

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponseDto> handleResourceNotFoundException(
    ResourceNotFoundException ex,
    HttpServletRequest request
) {
    ErrorResponseDto errorResponse = new ErrorResponseDto(
        LocalDateTime.now(),
        HttpStatus.NOT_FOUND.value(),
        HttpStatus.NOT_FOUND.getReasonPhrase(),
        ex.getMessage(),
        request.getRequestURI()
    );

    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
        .body(errorResponse);
}
```

Required import:

```
import jakarta.servlet.http.HttpServletRequest;
```

Validation Errors

39. Handling Validation Errors Globally

Suppose we have this request DTO:

```
public class CreateStudentRequestDto {

    @NotBlank(message = "Name cannot be blank")
    @Size(min = 2, max = 50, message = "Name must be between
2 and 50 characters")
    private String name;

    @NotBlank(message = "Email cannot be blank")
    @Email(message = "Email must be valid")
    private String email;

    @NotNull(message = "Age is required")
    @Min(value = 18, message = "Student must be at least 18 y
ears old")
    private Integer age;

    // getters and setters
}
```

If the client sends invalid data:

```
{
  "name": "",
  "email": "wrong-email",
  "age": 15
}
```

validation should fail and the API should return a clean `400 Bad Request` response.

40. Validation Flow in Spring Boot

The flow is:

```
Request comes
↓
Spring binds JSON to DTO
↓
Validation runs because of @Valid
↓
Validation fails
↓
Spring throws MethodArgumentNotValidException
↓
GlobalExceptionHandler catches it
↓
Field-level validation response is returned
```

Important point:

Validation runs only when we use `@Valid` in the controller.

Example:

```
@PostMapping
public ResponseEntity<StudentResponseDto> createStudent(
    @Valid @RequestBody CreateStudentRequestDto requestDt
o
) {
    StudentResponseDto savedStudent = studentService.createSt
udent(requestDto);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(savedStudent);
}
```

41. What Is `MethodArgumentNotValidException` ?

`MethodArgumentNotValidException` occurs when validation fails on a controller method argument.

Most common case:

```
@Valid @RequestBody CreateStudentRequestDto requestDto
```

If the request body violates validation rules, Spring throws

`MethodArgumentNotValidException`.

42. Field-Level Validation Error Response

A professional validation error response should tell exactly which field failed and why.

Example:

```
{
  "timestamp": "2026-07-07T11:20:30",
  "statusCode": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/students",
  "fieldErrors": {
    "name": "Name cannot be blank",
    "email": "Email must be valid",
    "age": "Student must be at least 18 years old"
  }
}
```

43. ValidationErrorResponseDto

```
package in.strikes.dtoDemo.dto;

import java.time.LocalDateTime;
import java.util.Map;
```

```
public class ValidationErrorResponseDto {

    private LocalDateTime timestamp;
    private int statusCode;
    private String error;
    private String message;
    private String path;
    private Map<String, String> fieldErrors;

    public ValidationErrorResponseDto(
        LocalDateTime timestamp,
        int statusCode,
        String error,
        String message,
        String path,
        Map<String, String> fieldErrors
    ) {
        this.timestamp = timestamp;
        this.statusCode = statusCode;
        this.error = error;
        this.message = message;
        this.path = path;
        this.fieldErrors = fieldErrors;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public int getStatusCode() {
        return statusCode;
    }

    public String getError() {
        return error;
    }
}
```

```
}

public String getMessage() {
    return message;
}

public String getPath() {
    return path;
}

public Map<String, String> getFieldErrors() {
    return fieldErrors;
}
}
```

44. Extracting Field Errors

Use `getBindingResult().getFieldErrors()`:

```
Map<String, String> fieldErrors = new HashMap<>();

ex.getBindingResult().getFieldErrors().forEach(error ->
    fieldErrors.put(error.getField(), error.getDefaultMes-
sage())
);
```

This creates a map like:

```
{
    "name": "Name cannot be blank",
    "email": "Email must be valid",
    "age": "Student must be at least 18 years old"
}
```

45. Handler for Validation Errors

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ValidationErrorResponseDto> handleValidationException(
    MethodArgumentNotValidException ex,
    HttpServletRequest request
) {
    Map<String, String> fieldErrors = new HashMap<>();

    ex.getBindingResult().getFieldErrors().forEach(error ->
        fieldErrors.put(error.getField(), error.getDefaultMessage())
    );

    ValidationErrorResponseDto errorResponse = new ValidationErrorResponseDto(
        LocalDateTime.now(),
        HttpStatus.BAD_REQUEST.value(),
        HttpStatus.BAD_REQUEST.getReasonPhrase(),
        "Validation failed",
        request.getRequestURI(),
        fieldErrors
    );

    return ResponseEntity
        .status(HttpStatus.BAD_REQUEST)
        .body(errorResponse);
}
```

Required imports:

```
import org.springframework.web.bind.MethodArgumentNotValidException;
```

```
import java.util.HashMap;  
import java.util.Map;
```

Bad Request Scenarios

46. What Is **400 Bad Request** ?

400 Bad Request means the client sent something wrong.

Validation failure is one type of bad request.

Other examples:

- Invalid JSON
- Wrong data type in JSON
- Malformed request body
- Invalid request format
- Invalid path variable type
- Missing required query parameter

47. Invalid JSON or Wrong Data Type

Suppose the DTO expects:

```
private Integer age;
```

But the client sends:

```
{  
  "name": "Amit",  
  "email": "amit@gmail.com",  
  "age": "twenty"  
}
```

Spring tries to convert `"twenty"` into `Integer`.

It cannot, so the API should return:

```
400 Bad Request
```

For unreadable or malformed request bodies, Spring throws:

```
HttpMessageNotReadableException
```

48. Handler for Invalid Request Body

```
@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ErrorResponseDto> handleHttpMessageNotR
eadeableException(
    HttpMessageNotReadableException ex,
    HttpServletRequest request
) {
    ErrorResponseDto errorResponse = new ErrorResponseDto(
        LocalDateTime.now(),
        HttpStatus.BAD_REQUEST.value(),
        HttpStatus.BAD_REQUEST.getReasonPhrase(),
        "Invalid request body. Please check your JSON for
mat and data types.",
        request.getRequestURI()
    );

    return ResponseEntity
        .status(HttpStatus.BAD_REQUEST)
        .body(errorResponse);
}
```

Required import:

```
import org.springframework.http.converter.HttpMessageNotReadableException;
```

49. Invalid Path Variable Type

Suppose the controller expects:

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
    Student student = studentService.getStudentById(id);
    return ResponseEntity.ok(student);
}
```

But the client calls:

```
GET /api/students/abc
```

Spring cannot convert `"abc"` into `Long`.

In this case, Spring may throw:

`MethodArgumentTypeMismatchException`

A clean API should return `400 Bad Request`.

Example handler:

```
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<ErrorResponseDto> handleMethodArgumentTypeMismatchException(
    MethodArgumentTypeMismatchException ex,
    HttpServletRequest request
) {
    ErrorResponseDto errorResponse = new ErrorResponseDto(
```

```

        LocalDateTime.now(),
        HttpStatus.BAD_REQUEST.value(),
        HttpStatus.BAD_REQUEST.getReasonPhrase(),
        "Invalid value for parameter '" + ex.getName() +
        "'. Please check the required data type.",
        request.getRequestURI()
    );

    return ResponseEntity
        .status(HttpStatus.BAD_REQUEST)
        .body(errorResponse);
}

```

Required import:

```

import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

```

Duplicate Resource / Conflict Handling

50. Difference Between **400 Bad Request** and **409 Conflict**

First principle:

400 means the request itself is invalid.
 409 means the request may be valid, but it conflicts with existing server data.

Example:

Invalid email format	→ 400 Bad Request
Email already exists	→ 409 Conflict
Age sent as "twenty"	→ 400 Bad Request
Roll number already exists	→ 409 Conflict

51. Creating DuplicateResourceException

```
package in.strikes.dtoDemo.exception;

public class DuplicateResourceException extends RuntimeException {

    public DuplicateResourceException(String message) {
        super(message);
    }
}
```

52. Throwing DuplicateResourceException from Service

```
public StudentResponseDto createStudent(CreateStudentRequestDto requestDto) {
    boolean emailExists = studentRepository.existsByEmail(requestDto.getEmail());

    if (emailExists) {
        throw new DuplicateResourceException(
            "Student with email " + requestDto.getEmail()
            + " already exists"
        );
    }

    Student student = new Student();
    student.setName(requestDto.getName());
    student.setEmail(requestDto.getEmail());
    student.setAge(requestDto.getAge());

    Student savedStudent = studentRepository.save(student);
}
```

```
        return mapToResponseDto(savedStudent);
    }
}
```

53. Handler for Duplicate Resource

```
@ExceptionHandler(DuplicateResourceException.class)
public ResponseEntity<ErrorResponseDto> handleDuplicateResourceException(
    DuplicateResourceException ex,
    HttpServletRequest request
) {
    ErrorResponseDto errorResponse = new ErrorResponseDto(
        LocalDateTime.now(),
        HttpStatus.CONFLICT.value(),
        HttpStatus.CONFLICT.getReasonPhrase(),
        ex.getMessage(),
        request.getRequestURI()
    );

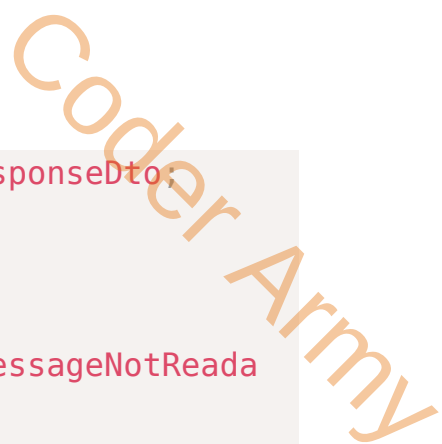
    return ResponseEntity
        .status(HttpStatus.CONFLICT)
        .body(errorResponse);
}
```

Final GlobalExceptionHandler Example

54. Complete Handler Class

```
package in.strikes.dtoDemo.exception;

import in.strikes.dtoDemo.dto.ErrorResponseDto;
```



```

import in.strikes.dtoDemo.dto.ValidationErrorResponseDto;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponseDto> handleResourceNotFoundException(
        ResourceNotFoundException ex,
        HttpServletRequest request
    ) {
        ErrorResponseDto errorResponse = new ErrorResponseDto(
            LocalDateTime.now(),
            HttpStatus.NOT_FOUND.value(),
            HttpStatus.NOT_FOUND.getReasonPhrase(),
            ex.getMessage(),
            request.getRequestURI()
        );
    }
}

```

```

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(errorResponse);
    }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ErrorResponseDto> handleDuplicateResourceException(
        DuplicateResourceException ex,
        HttpServletRequest request
    ) {
        ErrorResponseDto errorResponse = new ErrorResponseDto(
            LocalDateTime.now(),
            HttpStatus.CONFLICT.value(),
            HttpStatus.CONFLICT.getReasonPhrase(),
            ex.getMessage(),
            request.getRequestURI()
        );

        return ResponseEntity
            .status(HttpStatus.CONFLICT)
            .body(errorResponse);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ValidationErrorResponseDto> handleValidationException(
        MethodArgumentNotValidException ex,
        HttpServletRequest request
    ) {
        Map<String, String> fieldErrors = new HashMap<>();

        ex.getBindingResult().getFieldErrors().forEach(error

```

->

```

        fieldErrors.put(error.getField(), error.getDe
faultMessage())
    );

    ValidationErrorResponseDto errorResponse = new Valida
tionErrorResponseDto(
        LocalDateTime.now(),
        HttpStatus.BAD_REQUEST.value(),
        HttpStatus.BAD_REQUEST.getReasonPhrase(),
        "Validation failed",
        request.getRequestURI(),
        fieldErrors
    );

    return ResponseEntity
        .status(HttpStatus.BAD_REQUEST)
        .body(errorResponse);
}

@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ErrorResponseDto> handleHttpMessage
NotReadableException(
    HttpMessageNotReadableException ex,
    HttpServletRequest request
) {
    ErrorResponseDto errorResponse = new ErrorResponseDto
(
        LocalDateTime.now(),
        HttpStatus.BAD_REQUEST.value(),
        HttpStatus.BAD_REQUEST.getReasonPhrase(),
        "Invalid request body. Please check your JSON
format and data types.",
        request.getRequestURI()
    );

    return ResponseEntity

```

```

        .status(HttpStatus.BAD_REQUEST)
        .body(errorResponse);
    }

    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ErrorResponseDto> handleMethodArgumentTypeMismatchException(
        MethodArgumentTypeMismatchException ex,
        HttpServletRequest request
    ) {
        ErrorResponseDto errorResponse = new ErrorResponseDto(
            LocalDateTime.now(),
            HttpStatus.BAD_REQUEST.value(),
            HttpStatus.BAD_REQUEST.getReasonPhrase(),
            "Invalid value for parameter '" + ex.getName() + "'. Please check the required data type.",
            request.getRequestURI()
        );

        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(errorResponse);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponseDto> handleGenericException(
        Exception ex,
        HttpServletRequest request
    ) {
        ErrorResponseDto errorResponse = new ErrorResponseDto(
            LocalDateTime.now(),
            HttpStatus.INTERNAL_SERVER_ERROR.value(),

```

```

        HttpStatus.INTERNAL_SERVER_ERROR.getReasonPhrase(),
        "Something went wrong. Please try again later.",
        request.getRequestURI()
    );

    return ResponseEntity
        .status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(errorResponse);
}
}

```

Final Mental Model

55. Categories of Exceptions

In this project, we handle two major categories:

1. Business Exceptions

These represent meaningful business problems.

Exception	Meaning	Status Code
<code>ResourceNotFoundException</code>	Requested resource does not exist	404 Not Found
<code>DuplicateResourceException</code>	Resource already exists or conflicts with data	409 Conflict

2. Request/Input Exceptions

These happen when the client sends incorrect data.

Exception	Meaning	Status Code
<code>MethodArgumentNotValidException</code>	Validation failed on DTO	400 Bad Request

Exception	Meaning	Status Code
<code>HttpMessageNotReadableException</code>	Invalid JSON or wrong data type in request body	400 Bad Request
<code>MethodArgumentTypeMismatchException</code>	Wrong type in path variable or request parameter	400 Bad Request

3. Unexpected Exceptions

These are fallback server-side errors.

Exception	Meaning	Status Code
<code>Exception</code>	Any unhandled unexpected error	500 Internal Server Error

56. Clean API Flow After Global Exception Handling

```

Client sends request
↓
Controller receives request
↓
Controller calls service
↓
Service performs business logic
↓
If success:
    Controller returns success response
↓
If failure:
    Service throws meaningful exception
↓
GlobalExceptionHandler catches exception
↓
ErrorResponseDto / ValidationErrorResponseDto is returned

```

57. Why This Makes APIs Professional

Global exception handling improves the API because:

- Controllers stay clean.
- Repeated error logic is removed.

- Service layer communicates failures clearly.
- Frontend receives consistent error responses.
- Status codes match the real meaning of the failure.
- Debugging becomes easier because error responses contain timestamp and path.
- Unexpected errors are handled safely through a generic fallback.

58. Best Practices

Do not return `null` for missing resources

Avoid this:

```
return studentRepository.findById(id).orElse(null);
```

Prefer this:

```
return studentRepository.findById(id)
    .orElseThrow(() -> new ResourceNotFoundException(
        "Student with id " + id + " not found"
    ));
```

Keep success logic in controller

Controller should mainly handle successful responses.

```
return ResponseEntity.ok(student);
```

Keep business checks in service

Examples:

- Student exists or not
- Email already exists

- Roll number already exists
- User is allowed to perform action or not

Keep error response logic in `GlobalExceptionHandler`

The global handler should decide:

- Status code
- Error message
- Error response structure

Use specific handlers before relying on generic fallback

Specific handlers give meaningful responses.

Generic handler should only be a final safety net.

Do not expose internal exception messages in production

Avoid sending raw exception messages for unexpected errors.

Bad:

```
.body(ex.getMessage())
```

Better:

```
.body("Something went wrong. Please try again later.")
```

This prevents leaking internal implementation details.

Final Summary

A professional Spring Boot API should clearly communicate both success and failure.

For success responses, use `ResponseEntity` to control status codes, headers, and body.

For failure responses, avoid writing repeated `if-else` logic inside controllers.

The cleaner approach is:

```
Service throws meaningful custom exceptions.  
GlobalExceptionHandler catches them.  
API returns consistent HTTP error responses.
```

This makes the backend cleaner, easier to maintain, and easier for frontend developers to consume.